

แบบฝึกหัดการออกแบบอัลกอริทึมแบบเวียนเกิดและการวิเคราะห์ Big-O

คำตอบและโปรแกรมภาษา Java

ทำตามหัวข้อที่กำหนดในเอกสารงาน ข้อ 1-6 และงานทดลองเปรียบเทียบประสิทธิภาพ

ข้อ 1 การกลับลำดับสตริง

โจทย์: กำหนดสตริง S ให้สร้างสตริงใหม่ที่มีลำดับตัวอักษรย้อนกลับจากสตริงเดิม ตัวอย่าง Input: pots&pans Output: snap&stop

1. คำอธิบายแนวคิด

อัลกอริทึมที่ 1 Recursive Algorithm: เริ่มจากตัวอักษรตัวสุดท้าย แล้วนำไปต่อหน้าผลลัพธ์จากการเรียกเมธอดกับสตริงส่วนที่เหลือ ทำซ้ำจนเหลือสตริงยาว 0 หรือ 1 ตัว

อัลกอริทึมที่ 2 Iterative Algorithm: ใช้ลูปเริ่มจากตำแหน่งสุดท้ายของสตริง แล้วลดตำแหน่งลงจนถึง 0 โดยใช้ StringBuilder เก็บผลลัพธ์

2. Pseudocode

Recursive Reverse(s)

```
ถ้า s ยาว 0 หรือ 1
    คืนค่า s
นำตัวอักษรสุดท้ายของ S มาต่อหน้าผลลัพธ์
เรียก Reverse(s ที่ตัดตัวสุดท้ายออก)
คืนค่าผลลัพธ์
```

Iterative Reverse(s)

```
สร้าง StringBuilder result ว่าง
i = ความยาวของ s - 1
ขณะที่ i >= 0
    append s[i] ลง result
    ลด i ลง 1
คืนค่า result เป็น String
```

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static String reverseRecursive(String s) {
    if (s == null || s.length() <= 1) {
        return s == null ? "" : s;
    }
    return s.charAt(s.length() - 1)
        + reverseRecursive(s.substring(0, s.length() - 1));
}
```

```
static String reverseIterative(String s) {
    if (s == null || s.length() <= 1) {
```

```

        return s == null ? "" : s;
    }
    StringBuilder result = new StringBuilder();
    for (int i = s.length() - 1; i >= 0; i--) {
        result.append(s.charAt(i));
    }
    return result.toString();
}

```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input	Recursive	Iterative
pots&pans	snap&stop	snap&stop
abc	cba	cba
12345	54321	54321
สตริงว่าง	""	""

5. การวิเคราะห์ Time Complexity

ขนาดสตริง n	Recursive	Iterative
10	ทำงานประมาณ n ครั้ง	ทำงาน n ครั้ง
100	ทำงานประมาณ n ครั้ง แต่มีการสร้าง String ใหม่	ทำงาน n ครั้ง
1,000	ยังเป็น n ขึ้น แต่เวลาจริงเพิ่มจากการสร้าง String	n ขึ้น
10,000	ประมาณ $O(n^2)$ จาก substring และ +	ประมาณ $O(n)$ ด้วย StringBuilder

Recursive มี Time Complexity โดยรวมประมาณ $O(n^2)$ เพราะแต่ละรอบมีการสร้าง substring และต่อ String ด้วยเครื่องหมาย + ส่วน Iterative ที่ใช้ StringBuilder มี $O(n)$ เพราะอ่านตัวอักษรทีละตัวและ append

6. การวิเคราะห์ Space Complexity

Recursive ใช้พื้นที่ที่ส่กจากการเรียกซ้ำ $O(n)$ และยังมี String ชั่วคราวจาก substring/การต่อ String ทำให้การใช้งานหน่วยความจำจริงสูงกว่าแบบรูป ส่วน Iterative ที่ใช้ StringBuilder ใช้พื้นที่ $O(n)$ สำหรับผลลัพธ์ และมีตัวแปรควบคุมเพียงเล็กน้อย

7. ข้อดีและข้อจำกัด

วิธี	ข้อดี	ข้อจำกัด
Recursive	โค้ดตรงกับแนวคิดเวียนเกิดและเขียนตามใจทัยได้ง่าย	ช้ากว่าเมื่อสตริงยาว ใช้ stack และสร้าง String หลายตัว
Iterative + StringBuilder	เร็วกว่าและใช้หน่วยความจำค่อนข้างเหมาะสม	โค้ดไม่ได้แสดงการเวียนเกิด

ผลกระทบของการต่อ String ด้วย +: String ใน Java แก่ค่าเดิมไม่ได้ ดังนั้นการต่อทีละตัวในลูปจะสร้าง String ใหม่หลายครั้ง ทำให้เวลารวมมีแนวโน้มเป็น $O(n^2)$ แต่ StringBuilder สามารถเพิ่มตัวอักษรลงในบัฟเฟอร์เดิมได้ จึงเหมาะกับงานนี้มากกว่า

8. สรุปว่าอัลกอริทึมใดเหมาะสมกว่า

ถ้าต้องการดูตัวอย่างการทำงานแบบเวียนเกิด ให้ใช้ **Recursive** แต่ถ้าต้องการใช้งานกับสตริงยาวและเน้นเวลา ควรใช้ **Iterative** ที่ใช้ **StringBuilder** เพราะมี **Time Complexity** ต่ำกว่า

ข้อ 2 การตรวจสอบ Palindrome

โจทย์: ตรวจสอบว่าสตริงอ่านจากซ้ายไปขวาและขวาไปซ้ายแล้วได้ข้อความเดียวกันหรือไม่ และให้ละเว้นตัวพิมพ์เล็ก/ใหญ่ ช่องว่าง และเครื่องหมายวรรคตอน

1. คำอธิบายแนวคิด

Reverse and Compare: ทำความสะอาดข้อความก่อน แล้วสร้างสตริงย้อนกลับ จากนั้นนำข้อความเดิมมาเปรียบเทียบกับข้อความย้อนกลับ ถ้าเท่ากันแสดงว่าเป็น Palindrome

Recursive Two-Pointer: เปรียบเทียบตัวอักษรด้านซ้ายสุดกับด้านขวาสุด ถ้าไม่เท่ากันตอบ false ทันที

ถ้าเท่ากันให้ขยับเข้าหากันแล้วเรียกเมธอดซ้ำจนกว่าตัวชี้จะพบกัน

2. Pseudocode

Reverse and Compare(s)

 clean = ลบช่องว่างและเครื่องหมาย พร้อมเปลี่ยนเป็นตัวพิมพ์เล็ก

 reverse = กลับลำดับของ clean

 ถ้า clean เท่ากับ reverse

 คืนค่า true

 ไม่เช่นนั้นคืนค่า false

Recursive Two-Pointer(s, left, right)

 clean = ทำความสะอาด s

 ถ้า left >= right

 คืนค่า true

 ถ้า clean[left] ไม่เท่ากับ clean[right]

 คืนค่า false

 เรียกเมธอดกับ left+1 และ right-1

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static boolean isPalindromeByReverse(String s) {  
    String clean = normalize(s);  
    String reversed = reverseIterative(clean);  
    return clean.equals(reversed);  
}
```

```
static boolean isPalindromeRecursive(String s, int left, int right) {  
    String clean = normalize(s);  
    if (clean.length() == 0) {  
        return true;  
    }  
}
```

```

    left = Math.max(0, left);
    right = Math.min(right, clean.length() - 1);
    if (left > right) {
        return true;
    }
    return isPalindromeRecursiveClean(clean, left, right);
}

private static boolean isPalindromeRecursiveClean(String s, int left, int right) {
    if (left >= right) {
        return true;
    }
    if (s.charAt(left) != s.charAt(right)) {
        return false;
    }
    return isPalindromeRecursiveClean(s, left + 1, right - 1);
}

```

ฟังก์ชัน **normalize** จะเก็บเฉพาะตัวอักษร/ตัวเลขและแปลงเป็นตัวพิมพ์เล็ก ทำให้ **A man, a plan, a canal: Panama** ได้ผลเป็น **true**

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input	Reverse and Compare	Recursive Two-Pointer
racecar	true	true
level	true	true
algorithm	false	false
gohangasalamiimalasagnahog	true	true
A man, a plan, a canal: Panama	true	true
Hello, World!	false	false

5. การวิเคราะห์ Time Complexity

Best case ของการเปรียบเทียบแบบ **Recursive** ถ้าเจอคู่แรกไม่ตรงกันจะหยุดทันทีในส่วนการเปรียบเทียบ จึงเป็น $O(1)$ สำหรับขั้นเปรียบเทียบ แต่เพราะโปรแกรมต้อง **normalize** สตริงก่อน เวลาารวมยังเป็น $O(n)$

Worst case เมื่อเป็น **Palindrome** หรือจุดที่ไม่ตรงกันอยู่เกือบกลางสตริง จะต้องเปรียบเทียบหลายคู่ จึงเป็น $O(n)$ ส่วน **Reverse and Compare** ต้องสร้าง **reverse** และเปรียบเทียบ จึงเป็น $O(n)$

6. การวิเคราะห์ Space Complexity

Reverse and Compare ใช้ $O(n)$ เพื่อเก็บสตริงที่ทำความสะอาดและสตริงย้อนกลับ ส่วน **Recursive Two-Pointer** ใช้ $O(n)$ สำหรับ **recursive call stack** และมี **String** ที่ **normalize** อีก 1 ตัว

7. ข้อดีและข้อจำกัด

วิธี	ข้อดี	ข้อจำกัด
Reverse and Compare	เข้าใจง่ายและเขียนไม่ซับซ้อน	ต้องสร้างสตริงย้อนกลับเพิ่ม
Recursive Two-Pointer	หยุดได้ทันทีเมื่อเจอคู่ไม่ตรงกันในส่วนตรวจสอบ	ใช้ stack และอาจเกิด StackOverflowError กับข้อมูลยาวมาก

8. สรุปว่าอัลกอริทึมใดเหมาะสมกว่า

สำหรับข้อมูลทั่วไป Reverse and Compare เข้าใจง่ายและทำงานได้ดี ถ้าต้องการใช้แนวคิดเวียนเกิดและหยุดเร็วเมื่อคู่ตัวอักษรไม่ตรงกัน ให้ใช้ Recursive Two-Pointer แต่ต้องระวังความลึกของการเรียกซ้ำ

ข้อ 3 การเปรียบเทียบจำนวนสระและพยัญชนะ

โจทย์: ตรวจสอบว่าสตริงภาษาอังกฤษมีจำนวนสระมากกว่าจำนวนพยัญชนะหรือไม่ โดยนับเฉพาะ a, e, i, o, u และไม่นับตัวเลข ช่องว่าง หรือเครื่องหมายพิเศษ

1. คำอธิบายแนวคิด

Recursive Counting: อ่านตัวอักษรทีละตำแหน่ง ถ้าเป็นสระเพิ่มตัวนับสระ ถ้าเป็นตัวอักษรภาษาอังกฤษที่ไม่ใช่สระเพิ่มตัวนับพยัญชนะ แล้วเรียกตัวเองกับตำแหน่งถัดไป

Iterative Counting: ใช้ลูปผ่านทุกตัวอักษร และเพิ่มตัวนับสระหรือพยัญชนะตามชนิดของตัวอักษร

2. Pseudocode

Recursive Counting(s, index, vowels, consonants)

```
ถ้า index เท่ากับความยาวของ s
    คืนค่า vowels > consonants
c = s[index]
ถ้า c เป็นตัวอักษรภาษาอังกฤษ
    ถ้า c เป็น a,e,i,o,u เพิ่ม vowels
    ไม่เช่นนั้นเพิ่ม consonants
เรียก Counting กับ index + 1
```

Iterative Counting(s)

```
vowels = 0
consonants = 0
วนทุกตัวอักษรใน s
    ถ้าเป็น a-z/A-Z
        ถ้าเป็นสระ เพิ่ม vowels
        ไม่เช่นนั้นเพิ่ม consonants
คืนค่า vowels > consonants
```

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static boolean hasMoreVowelsRecursive(String s) {
    if (s == null) {
        return false;
    }
    return hasMoreVowelsRecursiveHelper(s, 0, 0, 0);
}

private static boolean hasMoreVowelsRecursiveHelper(String s, int index,
    int vowels, int consonants) {
```

```

    if (index == s.length()) {
        return vowels > consonants;
    }
    char c = Character.toLowerCase(s.charAt(index));
    if (isEnglishLetter(c)) {
        if (isVowel(c)) {
            vowels++;
        } else {
            consonants++;
        }
    }
    return hasMoreVowelsRecursiveHelper(s, index + 1, vowels, consonants);
}

static boolean hasMoreVowelsIterative(String s) {
    if (s == null) {
        return false;
    }
    int vowels = 0;
    int consonants = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = Character.toLowerCase(s.charAt(i));
        if (isEnglishLetter(c)) {
            if (isVowel(c)) vowels++;
            else consonants++;
        }
    }
    return vowels > consonants;
}

```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input	Vowels	Consonants	Result
education	5	4	true
hello	2	3	false
AEIOU	5	0	true
abc123 !!	1	2	false
12345	0	0	false

5. การวิเคราะห์ Time Complexity

ทั้ง Recursive Counting และ Iterative Counting ต้องตรวจตัวอักษรทุกตำแหน่ง จึงมี Time Complexity เป็น $O(n)$

6. การวิเคราะห์ Space Complexity

Iterative ใช้พื้นที่เพิ่ม $O(1)$ เพราะเก็บแค่ตัวนับและตำแหน่ง ส่วน Recursive ใช้ recursive call stack $O(n)$

7. จำนวน recursive calls และความเสถียร StackOverflowError

สำหรับสตริงยาว n ตัวอักษร Recursive Counting จะมีการเรียกเมธอดประมาณ n ครั้ง และมี Base Case อีก 1 ครั้ง รวมประมาณ $n+1$ calls เมื่อข้อมูลไม่กว้าง ถ้าสตริงยาวมาก การเรียกซ้อนกันจำนวนมากอาจทำให้เกิด StackOverflowError ได้

8. ขนาดข้อมูลที่เหมาะสมและสรุป

ข้อมูลขนาดเล็กเหมาะกับ Recursive เพราะดูตามแนวคิดได้ง่าย ส่วนข้อมูลขนาดใหญ่ควรใช้ Iterative เพราะไม่ใช่ stack ตามจำนวนข้อมูลและลดโอกาส StackOverflowError

ข้อ 4 การจัดกลุ่มจำนวนคู่และจำนวนคี่

โจทย์: จัดตำแหน่งสมาชิกในอาร์เรย์ใหม่ให้จำนวนคู่อยู่ก่อนจำนวนคี่ โดยไม่จำเป็นต้องเรียงค่าภายในกลุ่ม

หมายเหตุ: ยึดตามเงื่อนไขของโจทย์ คือ จำนวนคู่อยู่ก่อนจำนวนคี่

1. คำอธิบายแนวคิด

Recursive Two-Pointer: ใช้ left และ right ถ้าซ้ายเป็นคู่เลื่อน left ถ้าขวาเป็นคี่เลื่อน right แต่ถ้าซ้ายเป็นคี่และขวาเป็นคู่ให้สลับกัน แล้วเรียกแบบเวียนเกิดกับช่วงที่เหลือ

Iterative Two-Pointer: ใช้หลักเดียวกัน แต่ใช้ while loop แทน recursive call

Extra Array: สร้างอาร์เรย์ใหม่ โดยอ่านอาร์เรย์เดิมรอบแรกเพื่อใส่เลขคู่ทั้งหมด และอ่านอีกรอบเพื่อใส่เลขคี่ทั้งหมด

วิธีนี้จึงรักษาลำดับเดิมของสมาชิกในแต่ละกลุ่ม

2. Pseudocode

Recursive Two-Pointer(a, left, right)

 ถ้า left >= right จบ

 ถ้า a[left] เป็นคู่

 เรียกกับ left + 1, right

 ถ้า a[right] เป็นคี่

 เรียกกับ left, right - 1

 ถ้าซ้ายเป็นคี่และขวาเป็นคู่

 สลับ a[left] กับ a[right]

 เรียกกับ left + 1, right - 1

Iterative Two-Pointer(a)

 left = 0, right = n - 1

 ขณะที่ left < right

 ถ้าซ้ายเป็นคู่ เลื่อน left

 ถ้าขวาเป็นคี่ เลื่อน right

 ถ้าซ้ายเป็นคี่และขวาเป็นคู่ สลับค่าแล้วเลื่อนทั้งสองตำแหน่ง

Extra Array(a)

 สร้าง result ขนาดเท่ากับ a

 วน a เพื่อใส่สมาชิกที่เป็นคู่ลง result

 วน a อีกครั้งเพื่อใส่สมาชิกที่เป็นคี่ลง result

 คืนค่า result

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static void rearrangeRecursive(int[] a, int left, int right) {
    if (a == null || left >= right) return;
    if (a[left] % 2 == 0) {
        rearrangeRecursive(a, left + 1, right);
    } else if (a[right] % 2 != 0) {
        rearrangeRecursive(a, left, right - 1);
    } else {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        rearrangeRecursive(a, left + 1, right - 1);
    }
}

static void rearrangeTwoPointer(int[] a) {
    if (a == null || a.length <= 1) return;
    int left = 0, right = a.length - 1;
    while (left < right) {
        if (a[left] % 2 == 0) left++;
        else if (a[right] % 2 != 0) right--;
        else {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
            left++;
            right--;
        }
    }
}

static int[] rearrangeExtraArray(int[] a) {
    if (a == null) return new int[0];
    int[] result = new int[a.length];
    int index = 0;
    for (int value : a) if (value % 2 == 0) result[index++] = value;
    for (int value : a) if (value % 2 != 0) result[index++] = value;
    return result;
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input	Recursive / Iterative (ตัวอย่าง)	Extra Array
[7, 2, 9, 4, 1, 6, 3, 8]	[8, 2, 6, 4, 1, 9, 3, 7]	[2, 4, 6, 8, 7, 9, 1, 3]
[5, 2, 7, 4, 9, 6]	[6, 2, 4, 7, 9, 5]	[2, 4, 6, 5, 7, 9]

5. Time Complexity และ Space Complexity

วิธี	Time	Space
Recursive Two-Pointer	$O(n)$	$O(n)$ จาก call stack
Iterative Two-Pointer	$O(n)$	$O(1)$ เพิ่มเติม

Extra Array	$O(n)$	$O(n)$ จากอาร์เรย์ใหม่
-------------	--------	------------------------

6. จำนวนครั้งของการสลับข้อมูล

Two-Pointer ทั้งสองวิธีสลับข้อมูลเฉพาะเมื่อซ้ายเป็นคี่และขวาเป็นคู่ จำนวนการสลับมากที่สุดประมาณ $\text{floor}(n/2)$ ครั้ง ส่วน Extra Array ไม่ต้องสลับข้อมูลเลย

7. การเปลี่ยนแปลงอาร์เรย์เดิมและความเป็น Stable

Recursive Two-Pointer และ Iterative Two-Pointer เปลี่ยนแปลงอาร์เรย์เดิมโดยตรงและไม่เป็น Stable เพราะการสลับอาจทำให้ลำดับเดิมในกลุ่มเปลี่ยนได้ ส่วน Extra Array สร้างอาร์เรย์ใหม่และรักษาลำดับเดิมของสมาชิก จึงเป็น Stable

8. สรุปว่าอัลกอริทึมใดเหมาะสมกว่า

ถ้าต้องการประหยัดหน่วยความจำและไม่สนใจลำดับภายในกลุ่ม ให้ใช้ Iterative Two-Pointer แต่ถ้าต้องการรักษาลำดับเดิมของสมาชิก ควรใช้ Extra Array

ข้อ 5 การแบ่งอาร์เรย์ตามค่า k

โจทย์: จัดให้สมาชิกที่มีค่าน้อยกว่าหรือเท่ากับ k อยู่ด้านหน้า และสมาชิกที่มากกว่า k อยู่ด้านหลัง

1. คำอธิบายแนวคิด

Recursive Partition: ใช้ left และ right ค้นหาที่อยู่คิดฝั่ง ถ้าพบให้สลับ แล้วเรียกเมธอดกับช่วงที่เหลือ

Iterative Partition: ใช้ดูปและตัวชี้สองตำแหน่งทำเหมือนวิธี Recursive แต่ไม่มี call stack

Sorting-Based: เรียงอาร์เรย์ก่อน แล้วสมาชิกที่ $\leq k$ จะมาอยู่ด้านหน้าเอง ส่วนค่าที่ $> k$ จะอยู่ด้านหลัง

2. Pseudocode

Recursive Partition(a, k, left, right)

 ถ้า left $\geq$ right จบ

 เลื่อน left จนเจอสมาชิกที่ $> k$

 เลื่อน right จนเจอสมาชิกที่ $\leq k$

 ถ้า left < right

 สลับสองค่า

 เลื่อน left และ right

 เรียกเมธอดกับช่วงที่เหลือ

Iterative Partition(a, k)

 left = 0, right = n - 1

 ขณะที่ left < right

 หาอันแรกจากซ้ายที่ $> k$

 หาอันแรกจากขวาที่ $\leq k$

 ถ้ายังไม่ข้ามกัน ให้สลับค่า

Sorting-Based(a, k)

เรียง a จากน้อยไปมาก

สมาชิก $\leq k$ จะอยู่ด้านหน้า

สมาชิก $> k$ จะอยู่ด้านหลัง

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static void partitionRecursive(int[] a, int k, int left, int right) {  
    if (a == null || left >= right) return;  
    while (left < right && a[left] <= k) left++;  
    while (left < right && a[right] > k) right--;  
    if (left < right) {  
        int temp = a[left];  
        a[left] = a[right];  
        a[right] = temp;  
        left++;  
        right--;  
    }  
    partitionRecursive(a, k, left, right);  
}
```

```
static void partitionIterative(int[] a, int k) {  
    if (a == null || a.length <= 1) return;  
    int left = 0, right = a.length - 1;  
    while (left < right) {  
        while (left < right && a[left] <= k) left++;  
        while (left < right && a[right] > k) right--;  
        if (left < right) {  
            int temp = a[left];  
            a[left] = a[right];  
            a[right] = temp;  
            left++;  
            right--;  
        }  
    }  
}
```

```
static void partitionBySorting(int[] a, int k) {  
    if (a == null || a.length <= 1) return;  
    Arrays.sort(a);  
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

Input	k	Recursive / Iterative	Sorting-Based
[12, 4, 7, 15, 3, 10, 8]	8	[8, 4, 7, 3, 15, 10, 12]	[3, 4, 7, 8, 10, 12, 15]
[5, 1, 9, 4, 12, 2]	5	[2, 1, 4, 5, 12, 9]	[1, 2, 4, 5, 9, 12]

5. Time Complexity และ Space Complexity

วิธี	Time	Space	In-place
Recursive Partition	$O(n)$	$O(n)$ worst case จาก stack	ได้
Iterative Partition	$O(n)$	$O(1)$	ได้
Sorting-Based	ประมาณ $O(n \log n)$	ขึ้นกับการ sort แต่ทำกับอาร์เรย์เดิมได้	ได้

6. เหตุผลที่การเรียงลำดับอาจช้ากว่าที่จำเป็น

ปัญหานี้ต้องการเพียงแบ่งข้อมูลเป็นสองฝั่งตาม k จึงไม่จำเป็นต้องรู้ลำดับของสมาชิกภายในแต่ละฝั่ง การใช้ Partition สามารถทำได้ $O(n)$ แต่การเรียงอาร์เรย์ต้องทำงานเพิ่มเพื่อจัดลำดับทั้งหมด ทำให้โดยทั่วไปใช้เวลาประมาณ $O(n \log n)$ ซึ่งมากกว่าที่จำเป็น

7. ความสัมพันธ์กับ Partition ใน Quick Sort

แนวคิดเหมือนกับขั้นตอน Partition ของ Quick Sort คือใช้ตัวชี้จากซ้ายและขวา ค้นหาสมาชิกที่อยู่ฝั่งด้านแล้วสลับกัน เมื่อจบจะได้สมาชิกที่อยู่ด้านหนึ่งของจุดแบ่งตามเงื่อนไข ส่วน Quick Sort จะนำแนวคิดนี้ไปทำต่อแบบแบ่งช่วงเพื่อเรียงทั้งอาร์เรย์

8. สรุป

Recursive Partition และ Iterative Partition เหมาะเมื่อสนใจเฉพาะการแบ่งตาม k เพราะทำงาน $O(n)$ และเป็น In-place โดย Iterative ประหยัด stack มากกว่า ส่วน Sorting-Based เหมาะเมื่อหลังจากแบ่งแล้วต้องการอาร์เรย์ที่เรียงด้วย แต่ถ้าแค่ต้องแบ่งตาม k จะเสียเวลาเกินความจำเป็น

ข้อ 6 การค้นหาจำนวนที่มีผลรวมเท่ากับ k

โจทย์: กำหนดอาร์เรย์ A ที่เรียงจากน้อยไปมากและมีสมาชิกไม่ซ้ำกัน ให้หาคู่สมาชิกสองค่าที่มีผลรวมเท่ากับ k

1. คำอธิบายแนวคิด

Brute Force: ลองจับคู่สมาชิกทุกคู่ที่เป็นไปได้ แล้วตรวจว่าผลรวมเท่ากับ k หรือไม่

Recursive Two-Pointer: ใช้ $left=0$ และ $right=n-1$ ถ้าผลรวมต่ำกว่า k ให้เพิ่ม $left$ ถ้าสูงกว่า k ให้ลด $right$ ถ้าเท่ากันให้ตอบว่าพบ

Binary Search: เลือก $A[i]$ ทีละตัว แล้วค้นหา $k-A[i]$ ในช่วงที่เหลือด้วย Binary Search

2. Pseudocode

```
Brute Force(a, k)
  สำหรับ i จาก 0 ถึง n-2
    สำหรับ j จาก i+1 ถึง n-1
      ถ้า a[i] + a[j] == k คืนค่า true
    คืนค่า false
```

Recursive Two-Pointer(a, k, left, right)

ถ้า left >= right คืนค่า false

sum = a[left] + a[right]

ถ้า sum == k คืนค่า true

ถ้า sum < k เรียกด้วย left+1

ถ้า sum > k เรียกด้วย right-1

Binary Search(a, k)

สำหรับ i จาก 0 ถึง n-2

target = k - a[i]

ค้น target ในช่วง i+1 ถึง n-1 ด้วย Binary Search

ถ้าพบ คืนค่า true

คืนค่า false

3. โปรแกรมภาษา Java ที่เกี่ยวข้อง

```
static boolean findPairBruteForce(int[] a, int k) {  
    if (a == null || a.length < 2) return false;  
    for (int i = 0; i < a.length - 1; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] + a[j] == k) return true;  
        }  
    }  
    return false;  
}
```

```
static boolean findPairRecursive(int[] a, int k, int left, int right) {  
    if (a == null || left >= right) return false;  
    int sum = a[left] + a[right];  
    if (sum == k) return true;  
    if (sum < k) return findPairRecursive(a, k, left + 1, right);  
    return findPairRecursive(a, k, left, right - 1);  
}
```

```
static boolean findPairBinarySearch(int[] a, int k) {  
    if (a == null || a.length < 2) return false;  
    for (int i = 0; i < a.length - 1; i++) {  
        int target = k - a[i];  
        int index = binarySearch(a, target, i + 1, a.length - 1);  
        if (index != -1) return true;  
    }  
    return false;  
}
```

4. ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

A	k	ผลลัพธ์
[2, 4, 7, 11, 15, 20]	18	พบคู่ 7 กับ 11
[1, 3, 5, 8, 10]	9	Pair found: 1 and 8
[2, 4, 7, 11]	20	ไม่พบคู่

5. Time Complexity และ Space Complexity

อัลกอริทึม	Time Complexity	Space Complexity
Brute Force	$O(n^2)$	$O(1)$
Recursive Two-Pointer	$O(n)$	$O(n)$ จาก recursive call stack
Binary Search	$O(n \log n)$	$O(1)$

6. เหตุผลที่ Two-Pointer ใช้กับอาร์เรย์ที่เรียงแล้ว

เพราะอาร์เรย์เรียงจากน้อยไปมาก ถ้าคู่ซ้าย+ขวามีน้อยกว่า k เรารู้ได้ว่าต้องขยับ left ไปทางขวาเพื่อเพิ่มผลรวม และถ้าผลรวมมากกว่า k ก็ขยับ right ไปทางซ้ายเพื่อลดผลรวม ถ้าอาร์เรย์ยังไม่เรียง การขยับตัวแบบนี้ไม่รับประกันว่าจะไม่พลาดคู่ที่ต้องการ จึงใช้หลักนี้ไม่ได้โดยตรง

7. สรุปว่าอัลกอริทึมใดเหมาะสมกว่า

ถ้าอาร์เรย์เรียงแล้ว วิธี Recursive Two-Pointer มี Time Complexity ดีที่สุดคือ $O(n)$ แต่ต้องแลกกับ stack ถ้าต้องการประหยัดหน่วยความจำควรใช้ Two-Pointer แบบลูป ในโจทย์ที่มีข้อมูลเรียงแล้ว Binary Search ยังดีกว่า Brute Force แต่ช้ากว่า Two-Pointer ตาม Big-O

งานทดลองเปรียบเทียบประสิทธิภาพ

เลือกทดลอง 2 ข้อ คือ ข้อ 3 และข้อ 5 ตามเงื่อนไขให้มีข้อมูล 4 ระดับ และทดลองแต่ละขนาด 5 ครั้ง จากนั้นใช้ค่าเฉลี่ยของเวลาเป็นหน่วย nanoseconds

ข้อมูลทดลองข้อ 3 ใช้สตริงที่เป็น a และ b สลับกันเพื่อให้ต้องตรวจทุกตำแหน่ง ส่วนข้อ 5 ใช้อาร์เรย์ที่ค่ามากกว่าและน้อยกว่า k สลับกัน เพื่อให้มีการแบ่งข้อมูลจริง

ตารางทดลองข้อ 3

n	Recursive Counting เฉลี่ย (ns)	Iterative Counting เฉลี่ย (ns)
100	40072	36439
1,000	166572	124928
10,000	StackOverflowError	1502968
100,000	StackOverflowError	2197268

ตารางทดลองข้อ 5

n	Recursive Partition เฉลี่ย (ns)	Iterative Partition เฉลี่ย (ns)	Sorting-Based เฉลี่ย (ns)
100	4076	3099	7625
1,000	44094	31853	35216
10,000	55152	293334	344984
100,000	StackOverflowError	1096134	1966856

หมายเหตุ: เวลาเป็นผลจากการรันโปรแกรมครั้งนี้นับบนเครื่องทดลอง จึงสามารถแตกต่างกันได้ตามเครื่องและสภาพการทำงานของ Java

คำตอบคำถามการทดลอง

- ผลการทดลองสอดคล้องกับ Big-O ที่วิเคราะห์ไว้หรือไม่? ตอบ: โดยรวมสอดคล้องกัน คือ Iterative ที่เป็น $O(n)$ ยังสามารถทำงานกับ $n = 100,000$ ได้ ขณะที่ Recursive ที่มี stack โดตามจำนวนข้อมูลเกิด StackOverflowError และ Sorting-Based เพิ่มเวลามากกว่า Partition เมื่อข้อมูลใหญ่ขึ้น
- อัลกอริทึมใดเร็วที่สุดเมื่อข้อมูลมีขนาดเล็ก? ตอบ: จากผลทดลองของข้อ 5 Iterative Partition เร็วที่สุดที่ $n = 100$ และ 1,000 ในชุดทดลองนี้ ส่วนข้อ 3 Iterative Counting เร็วกว่าที่ $n = 100$ และ 1,000
- อัลกอริทึมใดเหมาะสมที่สุดเมื่อข้อมูลมีขนาดใหญ่? ตอบ: วิธี Iterative ที่มี $O(n)$ เหมาะกว่า เพราะไม่ใช้ recursive call stack และไม่เสี่ยง StackOverflowError แบบวิธี Recursive
- Recursive Algorithm มีข้อจำกัดด้านหน่วยความจำอย่างไร? ตอบ: ทุกการเรียกเมธอดจะเพิ่ม stack frame ถ้าข้อมูลยาวมาก stack จะเต็มและเกิด StackOverflowError ได้
- เหตุใดอัลกอริทึมที่มี Big-O เท่ากันจึงอาจใช้เวลาจริงต่างกัน? ตอบ: เพราะจำนวนคำสั่งภายในแต่ละรอบไม่เท่ากัน มีค่าใช้จ่ายจากการเรียกเมธอด การสร้างออบเจกต์ การจัดการหน่วยความจำ และผลของ JVM เช่น JIT ทำให้เวลาจริงต่างกัน
- การวัดเวลาเพียงครั้งเดียวเพียงพอหรือไม่? ตอบ: ไม่เพียงพอ เพราะเวลาอาจแกว่งจากการทำงานของเครื่องและ JVM จึงควรทดลองหลายครั้งแล้วหาค่าเฉลี่ย ตามที่ใบงานกำหนด 5 ครั้ง

การตรวจกรณีฐานและกรณีพิเศษที่ใช้ในโปรแกรม

- Recursive Reverse: Base Case คือสตริงยาว 0 หรือ 1 ตัว
- Recursive Palindrome: Base Case คือ $left \geq right$ และ Recursive Case คือเปรียบเทียบตัวอักษรแล้วขยับเข้าหากัน
- Recursive Vowel Counting: Base Case คือ index ถึงความยาวของสตริงแล้ว
- Recursive Rearrangement / Partition: Base Case คือช่วงข้อมูลหมดหรือ $left \geq right$
- Recursive Pair Search: Base Case คือ $left \geq right$

- สตริงว่างไม่ทำให้โปรแกรมหยุดผิดปกติ และอาร์เรย์ว่างหรือ **null** ถูกตรวจไว้ก่อนทำงาน

โปรแกรมภาษา **Java** ฉบับเต็มสำหรับส่งงาน

ไฟล์: Main.java

```
import java.util.Arrays;

public class Main {

    // ===== ข้อ 1 =====

    // Recursive Algorithm
    static String reverseRecursive(String s) {
        if (s == null || s.length() <= 1) {
            return s == null ? "" : s;
        }
        return s.charAt(s.length() - 1)
            + reverseRecursive(s.substring(0, s.length() - 1));
    }

    // Iterative Algorithm (ใช้ StringBuilder)
    static String reverseIterative(String s) {
        if (s == null || s.length() <= 1) {
            return s == null ? "" : s;
        }
        StringBuilder result = new StringBuilder();
        for (int i = s.length() - 1; i >= 0; i--) {
            result.append(s.charAt(i));
        }
        return result.toString();
    }

    // วิธีเสริมสำหรับทดลองผลของการต่อ String ด้วย +
    static String reverseIterativePlus(String s) {
        if (s == null || s.length() <= 1) {
            return s == null ? "" : s;
        }
        String result = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            result = result + s.charAt(i);
        }
        return result;
    }

    // ===== ข้อ 2 =====

    // ตัดช่องว่างและเครื่องหมายต่าง ๆ และไม่แยกตัวพิมพ์ใหญ่/เล็ก
    static String normalize(String s) {
        if (s == null) {
            return "";
        }
        StringBuilder clean = new StringBuilder();
```

```

    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (Character.isLetterOrDigit(c)) {
            clean.append(Character.toLowerCase(c));
        }
    }
    return clean.toString();
}

static boolean isPalindromeByReverse(String s) {
    String clean = normalize(s);
    String reversed = reverseIterative(clean);
    return clean.equals(reversed);
}

static boolean isPalindromeRecursive(String s, int left, int right) {
    String clean = normalize(s);
    if (clean.length() == 0) {
        return true;
    }
    // รอรับการส่งตำแหน่งจากสตริงเดิมที่ยังมีช่องว่างหรือเครื่องหมายอยู่
    left = Math.max(0, left);
    right = Math.min(right, clean.length() - 1);
    if (left > right) {
        return true;
    }
    return isPalindromeRecursiveClean(clean, left, right);
}

private static boolean isPalindromeRecursiveClean(String s, int left, int right) {
    // Base Case
    if (left >= right) {
        return true;
    }

    // Recursive Case
    if (s.charAt(left) != s.charAt(right)) {
        return false;
    }
    return isPalindromeRecursiveClean(s, left + 1, right - 1);
}

// ===== ข้อ 3 =====

static boolean hasMoreVowelsRecursive(String s) {
    if (s == null) {
        return false;
    }
    return hasMoreVowelsRecursiveHelper(s, 0, 0, 0);
}

private static boolean hasMoreVowelsRecursiveHelper(String s, int index,
                                                    int vowels, int consonants) {
    // Base Case
    if (index == s.length()) {

```



```

        return vowels > consonants;
    }

    char c = Character.toLowerCase(s.charAt(index));
    if (isEnglishLetter(c)) {
        if (isVowel(c)) {
            vowels++;
        } else {
            consonants++;
        }
    }
}

// Recursive Case
return hasMoreVowelsRecursiveHelper(s, index + 1, vowels, consonants);
}

static boolean hasMoreVowelsIterative(String s) {
    if (s == null) {
        return false;
    }
    int vowels = 0;
    int consonants = 0;

    for (int i = 0; i < s.length(); i++) {
        char c = Character.toLowerCase(s.charAt(i));
        if (isEnglishLetter(c)) {
            if (isVowel(c)) {
                vowels++;
            } else {
                consonants++;
            }
        }
    }
    return vowels > consonants;
}

private static boolean isEnglishLetter(char c) {
    return c >= 'a' && c <= 'z';
}

private static boolean isVowel(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
}

// ===== 4 =====

static void rearrangeRecursive(int[] a, int left, int right) {
    if (a == null || left >= right) {
        return;
    }

    if (a[left] % 2 == 0) {
        rearrangeRecursive(a, left + 1, right);
    } else if (a[right] % 2 != 0) {
        rearrangeRecursive(a, left, right - 1);
    }
}

```

```

    } else {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        rearrangeRecursive(a, left + 1, right - 1);
    }
}

```

```

static void rearrangeTwoPointer(int[] a) {
    if (a == null || a.length <= 1) {
        return;
    }

```

```

    int left = 0;
    int right = a.length - 1;

```

```

    while (left < right) {
        if (a[left] % 2 == 0) {
            left++;
        } else if (a[right] % 2 != 0) {
            right--;
        } else {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
            left++;
            right--;
        }
    }
}

```

```

static int[] rearrangeExtraArray(int[] a) {
    if (a == null) {
        return new int[0];
    }

```

```

    int[] result = new int[a.length];
    int index = 0;

```

```

    // ใส่เลขคู่ก่อน
    for (int value : a) {
        if (value % 2 == 0) {
            result[index++] = value;
        }
    }

```

```

    // แล้วใส่เลขคี่
    for (int value : a) {
        if (value % 2 != 0) {
            result[index++] = value;
        }
    }

```

```

    return result;
}

```

```
// ===== 5 =====
```

```
static void partitionRecursive(int[] a, int k, int left, int right) {  
    if (a == null || left >= right) {  
        return;  
    }  
  
    while (left < right && a[left] <= k) {  
        left++;  
    }  
  
    while (left < right && a[right] > k) {  
        right--;  
    }  
  
    if (left < right) {  
        int temp = a[left];  
        a[left] = a[right];  
        a[right] = temp;  
        left++;  
        right--;  
    }  
  
    partitionRecursive(a, k, left, right);  
}
```

```
static void partitionIterative(int[] a, int k) {  
    if (a == null || a.length <= 1) {  
        return;  
    }  
  
    int left = 0;  
    int right = a.length - 1;  
  
    while (left < right) {  
        while (left < right && a[left] <= k) {  
            left++;  
        }  
  
        while (left < right && a[right] > k) {  
            right--;  
        }  
  
        if (left < right) {  
            int temp = a[left];  
            a[left] = a[right];  
            a[right] = temp;  
            left++;  
            right--;  
        }  
    }  
}
```

```
static void partitionBySorting(int[] a, int k) {
```

```

    if (a == null || a.length <= 1) {
        return;
    }
    Arrays.sort(a);
}

// ===== 6 =====

static boolean findPairBruteForce(int[] a, int k) {
    if (a == null || a.length < 2) {
        return false;
    }

    for (int i = 0; i < a.length - 1; i++) {
        for (int j = i + 1; j < a.length; j++) {
            if (a[i] + a[j] == k) {
                return true;
            }
        }
    }
    return false;
}

static boolean findPairRecursive(int[] a, int k, int left, int right) {
    if (a == null || left >= right) {
        return false;
    }

    int sum = a[left] + a[right];
    if (sum == k) {
        return true;
    }

    if (sum < k) {
        return findPairRecursive(a, k, left + 1, right);
    }

    return findPairRecursive(a, k, left, right - 1);
}

static boolean findPairBinarySearch(int[] a, int k) {
    if (a == null || a.length < 2) {
        return false;
    }

    for (int i = 0; i < a.length - 1; i++) {
        int target = k - a[i];
        int index = binarySearch(a, target, i + 1, a.length - 1);
        if (index != -1) {
            return true;
        }
    }
    return false;
}

```

```

private static int binarySearch(int[] a, int target, int left, int right) {
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (a[mid] == target) {
            return mid;
        }
        if (a[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

// ===== แสดงตัวอย่าง =====

static void showExamples() {
    System.out.println("===== ตัวอย่างการทำงาน =====");

    String s1 = "pots&pans";
    System.out.println("ข้อ 1 Recursive: " + reverseRecursive(s1));
    System.out.println("ข้อ 1 Iterative: " + reverseIterative(s1));

    String s2 = "A man, a plan, a canal: Panama";
    System.out.println("ข้อ 2 Reverse and Compare: " + isPalindromeByReverse(s2));
    System.out.println("ข้อ 2 Recursive Two-Pointer: " +
        isPalindromeRecursive(s2, 0, s2.length() - 1));

    String s3 = "education";
    System.out.println("ข้อ 3 Recursive: " + hasMoreVowelsRecursive(s3));
    System.out.println("ข้อ 3 Iterative: " + hasMoreVowelsIterative(s3));

    int[] a4 = {7, 2, 9, 4, 1, 6, 3, 8};
    int[] a4r = a4.clone();
    int[] a4i = a4.clone();
    System.out.println("ข้อ 4 Input: " + Arrays.toString(a4));
    rearrangeRecursive(a4r, 0, a4r.length - 1);
    rearrangeTwoPointer(a4i);
    System.out.println("ข้อ 4 Recursive: " + Arrays.toString(a4r));
    System.out.println("ข้อ 4 Iterative: " + Arrays.toString(a4i));
    System.out.println("ข้อ 4 Extra Array: " +
        Arrays.toString(rearrangeExtraArray(a4)));

    int[] a5 = {12, 4, 7, 15, 3, 10, 8};
    int k5 = 8;
    int[] a5r = a5.clone();
    int[] a5i = a5.clone();
    int[] a5s = a5.clone();
    partitionRecursive(a5r, k5, 0, a5r.length - 1);
    partitionIterative(a5i, k5);
    partitionBySorting(a5s, k5);
    System.out.println("ข้อ 5 Input: " + Arrays.toString(a5) + ", k = " + k5);
}

```

```

System.out.println("ข้อ 5 Recursive: " + Arrays.toString(a5r));
System.out.println("ข้อ 5 Iterative: " + Arrays.toString(a5i));
System.out.println("ข้อ 5 Sorting: " + Arrays.toString(a5s));

int[] a6 = {2, 4, 7, 11, 15, 20};
int k6 = 18;
System.out.println("ข้อ 6 Input: " + Arrays.toString(a6) + ", k = " + k6);
System.out.println("ข้อ 6 Brute Force: " + findPairBruteForce(a6, k6));
System.out.println("ข้อ 6 Recursive Two-Pointer: " +
    findPairRecursive(a6, k6, 0, a6.length - 1));
System.out.println("ข้อ 6 Binary Search: " + findPairBinarySearch(a6, k6));
}

// ===== ทดลองจับเวลา =====

static long measureQ3Recursive(String s) {
    long start = System.nanoTime();
    hasMoreVowelsRecursive(s);
    long end = System.nanoTime();
    return end - start;
}

static long measureQ3Iterative(String s) {
    long start = System.nanoTime();
    hasMoreVowelsIterative(s);
    long end = System.nanoTime();
    return end - start;
}

static long measureQ5Recursive(int[] input, int k) {
    int[] a = input.clone();
    long start = System.nanoTime();
    partitionRecursive(a, k, 0, a.length - 1);
    long end = System.nanoTime();
    return end - start;
}

static long measureQ5Iterative(int[] input, int k) {
    int[] a = input.clone();
    long start = System.nanoTime();
    partitionIterative(a, k);
    long end = System.nanoTime();
    return end - start;
}

static long measureQ5Sorting(int[] input, int k) {
    int[] a = input.clone();
    long start = System.nanoTime();
    partitionBySorting(a, k);
    long end = System.nanoTime();
    return end - start;
}

static String makeTestString(int n) {

```

```

    StringBuilder s = new StringBuilder(n);
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            s.append('a');
        } else {
            s.append('b');
        }
    }
    return s.toString();
}

```

```

static int[] makePartitionTestArray(int n, int k) {
    int[] a = new int[n];
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            a[i] = k + 1;
        } else {
            a[i] = k - 1;
        }
    }
    return a;
}

```

```

static String averageQ3Recursive(String s) {
    long total = 0;
    int success = 0;
    for (int t = 1; t <= 5; t++) {
        try {
            total += measureQ3Recursive(s);
            success++;
        } catch (StackOverflowError e) {
            return "StackOverflowError";
        }
    }
    return String.valueOf(total / success);
}

```

```

static String averageQ3Iterative(String s) {
    long total = 0;
    for (int t = 1; t <= 5; t++) {
        total += measureQ3Iterative(s);
    }
    return String.valueOf(total / 5);
}

```

```

static String averageQ5Recursive(int[] a, int k) {
    long total = 0;
    int success = 0;
    for (int t = 1; t <= 5; t++) {
        try {
            total += measureQ5Recursive(a, k);
            success++;
        } catch (StackOverflowError e) {
            return "StackOverflowError";
        }
    }
}

```

```

    }
    return String.valueOf(total / success);
}

static String averageQ5Iterative(int[] a, int k) {
    long total = 0;
    for (int t = 1; t <= 5; t++) {
        total += measureQ5Iterative(a, k);
    }
    return String.valueOf(total / 5);
}

static String averageQ5Sorting(int[] a, int k) {
    long total = 0;
    for (int t = 1; t <= 5; t++) {
        total += measureQ5Sorting(a, k);
    }
    return String.valueOf(total / 5);
}

static void runExperiment() {
    int[] sizes = {100, 1000, 10000, 100000};

    System.out.println();
    System.out.println("==== Experiment 3 =====");
    System.out.println("n\tRecursive(ns)\tIterative(ns)");
    for (int n : sizes) {
        String s = makeTestString(n);
        System.out.println(n + "\t" + averageQ3Recursive(s) + "\t" + averageQ3Iterative(s));
    }

    System.out.println();
    System.out.println("==== Experiment 5 =====");
    System.out.println("n\tRecursive(ns)\tIterative(ns)\tSorting(ns)");
    for (int n : sizes) {
        int k = 500000;
        int[] a = makePartitionTestArray(n, k);
        System.out.println(n + "\t" + averageQ5Recursive(a, k) + "\t"
            + averageQ5Iterative(a, k) + "\t" + averageQ5Sorting(a, k));
    }
}

public static void main(String[] args) {
    showExamples();
    runExperiment();
}
}

```